PROJECT ABSTRACT

FinTrack: Automated Personal Financial Management Infrastructure

Domain: FinTech / Application Engineering

Architecture: MVC / Object-Oriented Design

Group Members:

1. AMAN UBAID SAID
2. ABIN V
3. MUHAMMED JUNAID P K
4. SHAHANA SHARIN KM
5. VISWANTH V

1. PROBLEM DESCRIPTION

Modern personal finance management is hindered by increasingly fragmented transaction ecosystems, where individuals navigate multiple disparate bank accounts, credit channels, and digital wallets. This architecture is designed to satisfy three core operational boundaries necessary for data-intensive software engineering: (i) **Data Ingestion:** It accepts considerable multi-faceted input datasets directly from users to replace inefficient bookkeeping. (ii) **Data Persistence:** Volatile run-time transaction logs are permanently committed to secure local relational databases or structured plain files to prevent data loss. (iii) **Computational Output:** The system programmatically cross-references dynamic active user records against long-term stored matrices to compile and display real-time financial health summaries.

2. SYSTEM FUNCTIONALITIES

- **High-Volume User Data Ingestion (Input):** Dynamic forms to capture multi-category expenses, credit line items, customized recurring monthly budget limits, and specific targeted financial milestone thresholds.
- **Persistent Storage Management (Storage):** Systematic record tracking committed directly within database management systems (SQLite/MySQL) or local structured flat-files (.dat/.csv) to maintain historical records securely.
- **Analytical Report Compilation (Output):** Automatic calculation of transactional velocity, monthly category run-rates, balance forecasts, and visual budget compliance bars rendered instantly on the graphical interface.

3. OBJECT-ORIENTED CLASS DESIGN

The core engine utilizes a modular, decoupled Object-Oriented design structure to map out clear data interactions:

- `User`: Manages demographic records, profile configurations, global budgeting metrics, and high-level targets.
- `Transaction`: Encapsulates individual transactional primitives (ID, timestamp, category, amount, payment mode).
- `BudgetManager`: Processes real-time run rates, evaluates threshold variations, and tracks category border non-compliance.
- `StorageEngine`: Abstract system layer orchestrating physical file read/write protocols or relational SQL CRUD pipelines to ensure long-term database persistence.
- `AnalyticsProcessor`: Executes statistical compilation and forecasting metrics across the stored historical data arrays.

4. SYSTEM REQUIREMENTS

Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Linux
- **Development Environment:** Python 3.9+ / Java JDK 17+
- **Database Systems:** SQLite / MySQL Server 8.0
- **Interface Framework:** Tkinter / JavaFX / Native Web Frameworks

Hardware Requirements:

- **Processor Architecture:** Intel Core i3 / AMD Ryzen 3 or higher
- **System Memory (RAM):** Minimum 4 GB RAM
- **Storage Capacity:** 500 MB minimum available disk space
- **Display Output:** 1024x768 resolution monitor unit